Die Wette: 1000 Zahlen, zehn Fragen

Binäre Suche — halbieren statt durchgehen

AUSGANGSSITUATION



Lea und Tim wetten um einen Döner. Lea: „Denk dir eine Zahl zwischen 1 und 1000. Ich finde sie mit **höchstens zehn Fragen** – du antwortest nur mit ‚größer', ‚kleiner' oder ‚richtig'." Tim: „Niemals. Das sind tausend Zahlen!" Lea hat noch eine zweite Idee: Mit demselben Trick will sie im Sekretariat prüfen, ob eine Schülerausweis-Nummer gültig ist. Die Liste der gültigen Nummern hat **1000 Einträge** und ist aufsteigend sortiert.

Wer gewinnt die Wette – und wie viele Vergleiche braucht Leas Trick in der Ausweisliste im schlimmsten Fall?

Was tippst du? Kreuze an, bevor du rechnest:

☐ Tim gewinnt ☐ Lea gewinnt ☐ kommt auf die Zahl an

Ausschnitt aus der sortierten Ausweisliste – 15 Nummern (gesetzte Annahme)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Nummer	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678

1 Aufgabe 1 — Aufwärmen

Nenne aus dem Gedächtnis die Anzahl der Vergleiche, die die lineare Suche aus der letzten Stunde in einer Liste mit 1000 Einträgen im schlimmsten Fall braucht.

SO GEHT'S

So sucht ein Rechner in einer sortierten Liste: die binäre Suche

1. Der Suchbereich ist zuerst die ganze Liste: **links** = 0, **rechts** = letzter Index.
2. Berechne die Mitte: mitte = (links + rechts) // 2 – dabei wird abgerundet.
3. Vergleiche den Wert an der Mitte mit dem gesuchten Wert: gleich → gefunden. Kleiner als gesucht → suche rechts weiter: links = mitte + 1. Größer als gesucht → suche links weiter: rechts = mitte − 1.
4. Wiederhole ab Schritt 2, solange links ≤ rechts gilt.
5. Wird links größer als rechts, ist der Bereich leer: Der Wert **fehlt**.

BEISPIEL

Beispiel: Suche nach 1349

Suche nach 1349. links 0, rechts 14, mitte 7 → 1701 ist größer als 1349, also rechts = 6. Neue Mitte (0 + 6) // 2 = 3 → 1349 gefunden. **2 Vergleiche** statt 4 bei der linearen Suche.

2 Aufgabe 2

Vervollständige das Protokoll für die Suche nach der Nummer **2417**. Der erste Schritt ist eingetragen. Trage darunter die Anzahl der Vergleiche und den Index ein.

Schritt	links	rechts	mitte	Nummer an mitte	kleiner / größer / gleich?	neuer Bereich
1	0	14	7	1701	kleiner als 2417	links = 8
2						
3						
4						
5						

Anzahl der Vergleiche: _____ 2417 steht bei Index: _____

3 Aufgabe 3

a) **Kennzeichne** für die Suche nach der Nummer **1900** in jedem Schritt den Bereich, der noch durchsucht wird (Rahmen um die Kästchen), und kreise die Mitte ein. Nicht gebrauchte Zeilen bleiben leer.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Schritt 1	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 2	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 3	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 4	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 5	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678

b) **Erkläre**, woran der Algorithmus erkennt, dass 1900 nicht in der Liste steht.

MERKE

Halbieren zählen

Jeder Vergleich **halbiert** den Suchbereich. Wie viele Vergleiche die binäre Suche höchstens braucht, verrät die Frage: Wie oft kann man n halbieren, bis nur noch ein Eintrag übrig ist? Suche dafür das kleinste k, für das $\mathbf{2^k \geq n}$ gilt.

$$\mathbf{2^4 = 16 \cdot 2^7 = 128 \cdot 2^{10} = 1024 \cdot 2^{20} \approx 1\ Million}$$

Beispiel: n = 15 Nummern: $2^4 = 16 \geq 15$, also höchstens 4 Vergleiche. n = 100: $2^7 = 128 \geq 100$, also höchstens 7.

4 Aufgabe 4

Berechne, wie viele Vergleiche die binäre Suche im schlimmsten Fall braucht – für Leas Zahlenwette (1000 Zahlen) und für eine Kundendatei mit 1 Million Einträgen. Trage zum Vergleich daneben ein, was die lineare Suche braucht.

n	kleinstes k mit $2^k \geq n$	binäre Suche: Vergleiche	lineare Suche: Vergleiche
1000			
1 000 000			

5  Aufgabe 5 — Partnerarbeit

Dokumentiere eine eigene Suche: Schreibe sieben Nummern **aufsteigend sortiert** auf und wähle einen gesuchten Wert – vorhanden oder nicht. Führe die binäre Suche durch und trage jeden Schritt ein. Dein Partner prüft die Mitte in jeder Zeile nach.

Index 0	1	2	3	4	5	6

Gesucht: _____

Schritt	links	rechts	mitte	Nummer an mitte	kleiner / größer / gleich?
1					
2					
3					

Vergleiche: _____ Index (oder „fehlt“): _____ ☐ vom Partner geprüft

INFO

Bausteine lesen und zusammensetzen

Eine Zeile mit Doppelpunkt am Ende öffnet einen **Block**: Alles, was dazugehört, steht eine Stufe tiefer (4 Leerzeichen). Eine `while`-Schleife läuft, solange ihre Bedingung wahr ist.

`//` ist die **Ganzzahl-Division**: `7 // 2` ergibt 3. So bleibt die Mitte ein gültiger Index. `return` beendet die Funktion sofort – deshalb kann nach dem Treffer-Block einfach mit `if` weitergeprüft werden.

6 Aufgabe 6 — Code bauen

Implementiere die Funktion `binaere_suche(liste, gesucht)` aus den 8 Bausteinen: Bringe sie in die richtige Reihenfolge und trage im Bauplan je Zeile den Buchstaben ein – in der Spalte seiner Einrückungsstufe. Jeder Baustein wird genau einmal gebraucht.

A
```
if liste[mitte] < gesucht:
    links = mitte + 1
```
Mitte zu klein: rechts weitersuchen

B
```
def binaere_suche(liste, gesucht):
```
Funktion: sortierte Liste und gesuchter Wert

C
```
return -1
```
Suchbereich leer: nicht gefunden

D
```
mitte = (links + rechts) // 2
```
Mitte berechnen, abgerundet

E
```
links = 0
rechts = len(liste) - 1
```
Suchbereich am Anfang: die ganze Liste

F
```
else:
    rechts = mitte - 1
```
Mitte zu groß: links weitersuchen

G
```
while links <= rechts:
```
solange der Suchbereich nicht leer ist

H
```
if liste[mitte] == gesucht:
    return mitte
```
Treffer: Index zurückgeben

Bauplan – Einrückungsstufe (eine Stufe = 4 Leerzeichen)

Zeile	Stufe 0	Stufe 1	Stufe 2	Stufe 3	Stufe 4
1					
2					
3					
4					
5					
6					
7					
8					

Wenn du nicht weiterkommst: Womit beginnt jede Funktion? Welche Zeilen enden mit einem Doppelpunkt – und was muss darunter eine Stufe tiefer stehen?

7 Aufgabe 7 — Schreibtischtest

Teste dein Programm aus Aufgabe 6 auf dem Papier mit `liste = [2, 5, 8, 12, 16]` und `gesucht = 12`: Trage die Werte der Variablen je Schleifendurchlauf ein und den Rückgabewert.

links	rechts	mitte	liste[mitte]	Welcher Baustein greift?

Rückgabewert: _____ Und mit `gesucht = 3`: Rückgabewert _____ nach _____ Vergleichen



Bevor du schreibst: Die Wette geht über **1000 Zahlen**, nicht über die 15 aus dem Ausschnitt. Welche Zeile aus Aufgabe 4 brauchst du dafür?

8 Aufgabe 8 — Die Wette

Begründe mit deinen Ergebnissen aus Aufgabe 4, wer die Wette gewinnt – und warum Leas Trick bei Mias unsortierter Ticketliste aus der letzten Stunde nicht funktioniert. Vergleiche am Ende mit deinem Tipp vom Anfang.

Satzanfänge: „… gewinnt, weil bei 1000 Zahlen höchstens …“ · „Nach jeder Frage bleibt nur noch …“ · „Bei einer unsortierten Liste weiß Lea nach der Antwort ‚größer‘ nicht, …“

9 Aufgabe 9 — Vertiefung

Beurteile, ob Lea die Wette noch gewinnen kann, wenn Tim die Regeln ändert: Er antwortet nur noch mit „ja“ oder „nein“ auf Fragen wie „Ist es die 512?“.

10 Aufgabe 10 — Zusatz

Entwickle ein Vorgehen für das Sekretariat: Die 1000 Ausweisnummern liegen dort in der Reihenfolge der Ausgabe vor – unsortiert. Die Sekretärin prüft am Tag etwa 500 Nummern. Halte fest, welche Schritte das Programm einmalig und welche es bei jeder Prüfung ausführen soll, und gib grob an, wie viele Vergleiche pro Tag dein Vorgehen spart.

PRÜFE DICH SELBST

- [ ] Ist die Mitte in deinem Protokoll aus Aufgabe 2 immer abgerundet – nie 11,5?
- [ ] Wird dein Bereich in Aufgabe 3 in jedem Schritt kleiner, bis er leer ist?
- [ ] Liefert deine Funktion für 1900 die Zahl −1 – und läuft nicht endlos?